

CO-BÜCHI BARRIER CERTIFICATES FOR DISCRETE-TIME DYNAMICAL SYSTEMS

VISHNU MURALI¹, ASHUTOSH TRIVEDI¹, AND MAJID ZAMANI¹

ABSTRACT. Barrier certificates provide functional overapproximations for the reachable set of dynamical systems and provide inductive guarantees on the safe evolution of the system. In automata-theoretic verification, a key query is to determine whether the system visits a given predicate over the states “finitely often”, resulting from the complement of the traditional Büchi acceptance condition. This paper proposes a barrier certificate approach to answer such queries by developing a notion of *co-Büchi barrier certificates* (CBBCs) that generalize classic barrier certificates to ensure that the traces of a system visit a given predicate a fixed number of times. Our notion of CBBC is inspired from *bounded synthesis* paradigm to LTL realizability, where the LTL specifications are converted to safety automata via (universal) co-Büchi automata with a bound on final state visitations provided as a hyperparameter. Our application of CBBCs in verification is analogous: we fix a bound and search for a suitable barrier certificate, increasing the bound if no suitable function can be found. We then use these CBBCs to verify our system against properties specified by co-Büchi automata and demonstrate their effectiveness via some case studies.

1. INTRODUCTION

The notion of *barrier certificates* [1] combined with constraint satisfaction based search approaches have transformed automatic safety verification for dynamical systems. A barrier certificate is a function over the state set of the system, such that the value of the barrier is non-positive over the initial set of states, is non-increasing as the system evolves, and positive over the unsafe set of states. They thus act as a functional overapproximation of the reachable set of states where any state is in the overapproximation if it has a value that is non-positive. When combined with a positivity condition over the unsafe set, it guarantees safety. In the classic automata-theoretic approach to verification, the central query involves ascertaining whether a set of states may only be visited finitely often. In particular, bounded synthesis based approaches [2, 3, 4, 5] are centered around the idea that the problem of determining whether a set is visited only finitely often can be answered by demonstrating a safety condition; this condition characterizes that the number of times the system can reach the aforementioned set is bounded from above by some fixed hyperparameter. *This paper presents an approach to automata-theoretic verification of discrete-time dynamical systems by advocating a notion of Co-Büchi barrier certificates.*

Contributions. The key idea of the paper builds around the following observation: while the question of visiting a region only finitely often is not a direct safety property for the system, if one instead considers a related system that appends the original system with a counter keeping track of the region visitation, a bounded-visitation property can be cast as a safety property over the extended system. Moreover, if the extended system is safe with respect to the modified safety property, it provides a certificate for bounded-visitation of the region for the original system. Of course, we need not explicitly compute this modified system; instead we may simply modify the definition of barrier certificates over the original system. We dub this modified definition *Co-Büchi barrier certificates* (CBBCs). We build on this idea, and propose an automata-theoretic technique to verify discrete-time dynamical systems against properties specified as universal co-Büchi automata (UCA). We present two computational methods (SMT and Sum-of-Squares based) to search for these CBBCs.

Corresponding author Vishnu Murali. This work was supported by the NSF grants ECCS-2015403, CNS-2039062, and CNS-2111688.

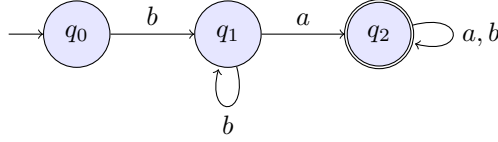


FIGURE 1. Example NBA $\mathcal{A}_{b_1}$ from Section 1 which represents the complement of a safety specification.

Related works. Traditional model checking approaches [6, 7, 8] for continuous space systems against temporal properties, such as those specified in linear temporal logic [9], typically involve building and verifying their finite-state abstractions against these properties. However, these approaches suffer from the curse of dimensionality: their complexity is exponential in the dimension of the state space. First proposed by Prajna et al. [10], the notion of barrier certificates has seen widespread use [11, 12, 13] in the verification of safety for continuous-space systems as they provide an abstraction-free approach. While these approaches are sound, techniques to find barrier certificates of a certain template (*e.g.* a polynomial) that rely on semidefinite programming [14] such as sum-of-squares [15], have a complexity that is polynomial in the dimension of the state space (assuming their template is fixed).

Wongpiromsarn et al. [16] introduced the idea of using barrier certificate to verify continuous-space systems against temporal logic properties and subsequently it has inspired several extensions [17, 18, 19, 20, 21]. A key feature of the approaches from [16, 20] is the idea of “cutting” the automaton (by breaking it into state triplets) using barrier certificates; we call this approach the state triplet approach in our work. As an example consider a system, a labeling function mapping states to observations, and a specification. The complement of the specification is shown in Figure 1 and our goal is to find a barrier certificate that cuts every path from the initial state to the accepting state by demonstrating that some sequences of transitions within this path is impossible. For instance, in Figure 1, we are required to “cut” the state triplet (q_0, q_1, q_2) to disallow all traces that start from any state with the label b from reaching a state with the label a . While the state triplet approach is useful in verifying properties specified by automaton on finite words (especially properties that are specified by co-safe languages [22]), it unfortunately faces limitations when dealing with general automata on infinite words. Note that the acceptance condition of automata over infinite words ask whether an accepting state is visited infinitely often, however this state triplet approach may be used to verify only those systems whose traces reach the accepting state of the automaton at most twice (cf. Subsection 6.1). Further this approach is independent of the runs in the automaton and of the initial states of the system, in the following way: one is required to break the edge pairs of a path regardless of what states of the automaton may be encountered before or after. We show (cf. Subsection 6) that our CBBC-based approach generalizes the state triplet approach for discrete-time dynamical systems. The authors of [23] present an approach to verify ω -regular properties via transition invariants rather than state invariants. However, the complexity of the certificates is dependent on twice the number of dimensions of the system. Our notion of CBBCs is inspired by the ideas present in bounded synthesis [2, 3, 4, 5], where one successively increases the bound on the number of times a region is visited. While the approach in [18] relies on the product of the system with the automaton and is thus less conservative in comparison to the state triplet approach, it does not allow the same degree of giving incremental guarantees, as one needs to unfold the automaton to increase the bound on the number of visits. The authors of [17] provide a notion of parity certificates to provide guarantees against parity automata via a combination of inductive invariants and ranking functions. CBBCs are not only sufficient but also necessary and rely only on safety arguments. Our notion of CBBCs have been extended to provide a compositional approach to synthesize controllers for large-scale systems represented as interconnections of smaller subsystems in [24] as well as for data-driven and stochastic systems [25, 26].

2. PRELIMINARIES

We use $\mathbb{N}$ and $\mathbb{R}$ to denote the set of natural numbers and reals. For $a \in \mathbb{R}$, we use $\mathbb{R}_{\geq a}$ and $\mathbb{R}_{> a}$ to denote the intervals $[a, \infty)$ and (a, ∞) respectively, and similarly, for any natural number $n \in \mathbb{N}$, we use $\mathbb{N}_{\geq n}$ to denote the set of natural numbers greater than or equal to n . We use $[0..k]$ to denote the set $\{0, \dots, k\}$ for any $k \in \mathbb{N}$. Given a set A , sets A^* and A^ω denote the set of finite and countably infinite sequences of elements in A , while $|A|$ denotes the cardinality of the set. Given two sets A, B , we use $A \setminus B$ to denote the set containing those elements that are present in A but not in B , and as usual use $A \cup B$ and $A \cap B$ to represent their union and intersection. If $A \subseteq B$, and the set B can be inferred from the context, we denote the complement $B \setminus A$ simply as $\bar{A}$.

Given a function $f : A \rightarrow A$, we define $f^i : A \rightarrow A$ recursively as $f^0(a) = a$ for any $a \in A$, and $f^{i+1}(a) = f^i(f(a))$ for all $i \in \mathbb{N}$. Given a relation $R \subseteq A \times B$, and an element $a \in A$, let $R(a)$ denote the set $\{b \mid (a, b) \in R\}$. We use $(a_1, a_2, \dots, a_n) \in A^*$ to indicate a finite-length sequence of length $n \in \mathbb{N}$, and $\langle a_0, a_1, \dots \rangle \in A^\omega$ to indicate a countably infinite sequence of elements from the set A . We say that an infinite sequence $s = \langle a_0, a_1, \dots \rangle \in A^\omega$, visits an element $a \in A$, at most n times if there exists at most n distinct indices $i_0, i_1, \dots, i_{(n-1)} \in \mathbb{N}$ with $a_{i_j} = a$ for all $j \in [0..(n-1)]$, and $a_\ell \neq a$ for all $\ell \in \mathbb{N} \setminus \{i_0, i_1, \dots, i_{(n-1)}\}$. This generalizes to determining whether a sequence s visits a subset $B \subseteq A$ at most n times. Let $\text{Inf}(s)$ be the set of elements that occur infinitely often in the sequence s .

2.1. Automata over Infinite Words. An automaton $\mathcal{A}$ is a tuple $(\Sigma, Q, Q_0, \delta, Q_{Acc})$, where Σ denotes a finite alphabet, Q a finite set of states, $Q_0 \subseteq Q$ an initial set of states, $\delta \subseteq Q \times \Sigma \times Q$ the transition relation, and $Q_{Acc} \subseteq Q$ denotes a set of accepting states. A run of the automaton $\mathcal{A} = (\Sigma, Q, q_0, \delta, Q_{Acc})$ over a word $w = \langle \sigma_0, \sigma_1, \sigma_2, \dots \rangle \in \Sigma^\omega$, is an infinite sequence of states $\rho = \langle q_0, q_1, q_2, \dots \rangle \in Q^\omega$ with $q_0 \in Q_0$ and $q_{i+1} \in \delta(q_i, \sigma_i)$. A *nondeterministic Büchi automaton* (NBA) $\mathcal{A}_b = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ is said to accept a word w , if there exists a run ρ on w where $\text{Inf}(\rho) \cap Q_{Acc} \neq \emptyset$. A *Universal Co-Büchi automaton* (UCA) $\mathcal{A}_c = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ accepts a word w , if for every run ρ on w we have $\text{Inf}(\rho) \cap Q_{Acc} = \emptyset$. Our definition of a UCA is based on the definition considered in [4], though alternative, and equivalent semantics exist [27]. For $k \in \mathbb{N}$, a *k-Universal Co-Büchi automaton* (*k-UCA*) $\mathcal{A}_{c,k} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ accepts a word w if, every run ρ on w visits some state in the set Q_{Acc} at most k times. The language of an automaton $\mathcal{A}$ denoted $L(\mathcal{A})$ is defined as the set of words accepted by the automaton, *i.e.*, $L(\mathcal{A}) = \{w \mid \mathcal{A} \text{ accepts word } w\}$. It is well known that NBAs are closed under complementation [28]: given an NBA $\mathcal{A}_b = (\Sigma, Q, Q_0, \delta, Q_{Acc})$, there exists an NBA $\mathcal{A}'_b = (\Sigma, Q', Q'_0, \delta', Q'_{Acc})$ such that $L(\mathcal{A}_b) = \bar{L}(\mathcal{A}'_b)$. We also observe that the acceptance conditions of an NBA and a UCA are duals of each other, *i.e.*, given an NBA $\mathcal{A}_b = (\Sigma, Q, Q_0, \delta, Q_{Acc})$, and a UCA $\mathcal{A}_c = (\Sigma, Q, Q_0, \delta, Q_{Acc})$, with the same structure (the same sets and transition relation), we have $L(\mathcal{A}_b) = \bar{L}(\mathcal{A}_c)$. Observe that for natural numbers $k_1, k_2 \in \mathbb{N}$, a k_1 -UCA $\mathcal{A}_{c,k_1} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$, and a k_2 -UCA $\mathcal{A}_{c,k_2} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ with the same structure, we have $L(\mathcal{A}_{c,k_1}) \subseteq L(\mathcal{A}_{c,k_2})$. Finally, for a k -UCA $\mathcal{A}_{c,k} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ and the UCA $\mathcal{A}_c = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ we have $L(\mathcal{A}_{c,k}) \subseteq L(\mathcal{A}_c)$.

2.2. Discrete-time Dynamical System. A discrete-time dynamical system (or simply, a system) $\mathfrak{S}$ is a tuple $(\mathcal{X}, \mathcal{X}_0, f)$, where $\mathcal{X} \subseteq \mathbb{R}^n$ denotes the state set, $\mathcal{X}_0 \subseteq \mathcal{X}$ denotes a set of initial states, and $f : \mathcal{X} \rightarrow \mathcal{X}$ the state transition function. The state evolution of the system is given by $\mathfrak{S} : x(t+1) = f(x(t))$.

A *state sequence* is an infinite sequence $\langle x_0, x_1, \dots \rangle \in \mathcal{X}^\omega$ where $x_0 \in \mathcal{X}_0$, and $x_{i+1} = f(x_i)$, for all $i \in \mathbb{N}$. We associate a labeling function $\mathcal{L} : \mathcal{X} \rightarrow \Sigma$ which maps each state with a letter in a finite alphabet. This generalizes to mapping a state sequence of the system $\langle x_0, x_1, \dots \rangle \in \mathcal{X}^\omega$ to a trace or word $w = \langle \mathcal{L}(x_0), \mathcal{L}(x_1), \dots \rangle \in \Sigma^\omega$. Finally, let $\text{Traces}(\mathfrak{S}, \mathcal{L})$ denote the set of all traces of system $\mathfrak{S}$ under the labeling map $\mathcal{L}$.

2.3. Safety Verification and Barrier Certificates. A system $\mathfrak{S}$ is safe with respect to a set of unsafe states $\mathcal{X}_u$, if no state sequence reaches $\mathcal{X}_u$, *i.e.*, for every state sequence $\langle x_0, x_1, \dots \rangle$, we have $x_i \notin \mathcal{X}_u$ for all $i \in \mathbb{N}$.

Definition 2.1. A function $\mathcal{B} : \mathcal{X} \rightarrow \mathbb{R}$ is a barrier certificate for a system $\mathfrak{S}$ with respect to a set of unsafe states $\mathcal{X}_u$ if:

- (1) $\mathcal{B}(x) \leq 0$, for all $x \in \mathcal{X}_0$,
- (2) $\mathcal{B}(x) > 0$, for all $x \in \mathcal{X}_u$, and
- (3) $\mathcal{B}(f(x)) - \mathcal{B}(x) \leq 0$, for all $x \in \mathcal{X}$.

The existence of a barrier certificate ensures the safety of a system as discussed below.

Theorem 2.2. Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, with a set of unsafe states $\mathcal{X}_u$. The existence of a function $\mathcal{B} : \mathcal{X} \rightarrow \mathbb{R}$ as in Definition 2.1 implies that the system is safe.

The proof of Theorem 2.2 follows from [10]. We should note that while barrier certificates are a sufficient proof of safety for continuous-time systems, they are not only sufficient but also necessary for discrete-time systems.

Lemma 2.3. Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ that is safe with respect to a set of unsafe states $\mathcal{X}_u$. Then there exists a barrier certificate $\mathcal{B} : \mathcal{X} \rightarrow \mathbb{R}$ as in Definition 2.1.

Proof. Consider an oracle function $\mathcal{O} : 2^{\mathcal{X}} \rightarrow 2^{\mathcal{X}}$ defined as $\mathcal{O}(S) = \{x' \mid \text{there exists } i \in \mathbb{N}, \text{ such that } x' = f^i(x) \text{ for some } x \in S\}$ for all sets $S \subseteq \mathcal{X}$. Observe that the set $\mathcal{O}(\mathcal{X}_0)$ denotes the set of all reachable states of the system. As the system is safe, we must have $\mathcal{O}(\mathcal{X}_0) \cap \mathcal{X}_u = \emptyset$. Consider the function $\mathcal{B} : \mathcal{X} \rightarrow \mathbb{R}$ defined

as: $\mathcal{B}(x) = \begin{cases} 0 & \text{if } x \in \mathcal{O}(\mathcal{X}_0) \\ 1 & \text{otherwise} \end{cases}$. We now show how this function is a barrier certificate as in Definition 2.1.

Observe that conditions (1) and (2) trivially hold as $\mathcal{X}_0 \in \mathcal{O}(\mathcal{X}_0)$ by definition and $\mathcal{X}_u \cap \mathcal{O}(\mathcal{X}_0) = \emptyset$ from the assumption that the system is safe. Now, let us consider (3). The only way for (3) to not hold, is if we have $\mathcal{B}(f(x)) > \mathcal{B}(x)$, in such a case we must have $\mathcal{B}(x) = 0$, and $\mathcal{B}(f(x)) = 1$. If $\mathcal{B}(f(x)) = 1$, then $f(x) \notin \mathcal{O}(\mathcal{X}_0)$, while $\mathcal{B}(x) = 0$ implies that $x \in \mathcal{O}(\mathcal{X}_0)$. If the state $x \in \mathcal{O}(\mathcal{X}_0)$, by the definition of oracle $\mathcal{O}$, there must exist $x_0 \in \mathcal{X}_0$, and $i \in \mathbb{N}$, such that $x = f^i(x_0)$. Furthermore $f(x) = f^{i+1}(x_0)$, and so we must have $f(x) \in \mathcal{O}(x)$. This is a contradiction. $\square$

We now consider the problem of verifying more complex ω -regular objectives via barrier certificates.

2.4. Problem Definition. We now focus on the problem of characterizing barrier certificates to demonstrate that a set of states is visited only finitely often, and use it to provide a barrier certificate based approach for automata-theoretic verification. Particularly, we wish to define effective notions of barrier certificates to demonstrate that every state sequence of $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ visits a region $\mathcal{X}_{VF} \subseteq \mathcal{X}$ only finitely often. We then use these certificates to verify systems against properties specified by automata.

Given an NBA $\mathcal{A}_b = (\Sigma, Q, Q_0, \delta, Q_{Acc})$, and UCA $\mathcal{A}_c = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ we have $L(\mathcal{A}_c) = \overline{L(\mathcal{A}_b)}$. Thus showing that every trace of the system $\mathfrak{S}$ visits Q_{Acc} only finitely often is sufficient to verify that the traces of the system are in $L(\mathcal{A}_c)$ and hence not in $L(\mathcal{A}_b)$.

Problem 2.4. Prove that the traces $TR(\mathfrak{S}, \mathcal{L})$ of $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ under labeling $\mathcal{L} : \mathcal{X} \rightarrow \Sigma$, are in the language of a k -UCA $\mathcal{A}_{c,k} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$.

Corollary 2.5. If the traces $TR(\mathfrak{S}, \mathcal{L})$ of a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ under a labeling map $\mathcal{L} : \mathcal{X} \rightarrow \Sigma$, are contained in the language of a k -UCA $\mathcal{A}_{c,k} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$, then they are contained in the language of the UCA $\mathcal{A}_c = (\Sigma, Q, Q_0, \delta, Q_{Acc})$.

3. CO-BÜCHI BARRIER CERTIFICATES (CBBCs)

Barrier certificates act as proofs for safety, however many properties of interest require showing that a given region is visited a finite number of times rather than never visited at all. In particular, the accepting conditions of NBAs and UCAs rely on showing that the accepting states are visited infinitely or only finitely often. We thus consider approaches where one may use barrier certificates to show a set $\mathcal{X}_{VF}$ is visited only finitely often.

Our notion of CBBCs are inspired by the bounded synthesis paradigm [4], where one fixes the number of times an accepting state is visited and tries to prove that the system does not visit an accepting state beyond this bound. Analogously, we fix a counter to keep track of the number of times a state in $\mathcal{X}_{VF}$ is seen. We then modify the system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ by appending this counter variable to the state set. Formally, we construct a system $\mathfrak{S}' = (\mathcal{X}', \mathcal{X}'_0, f')$, where $\mathcal{X}' = \mathcal{X} \times \mathbb{N}$ denotes the state set (containing an additional counter value). The set $\mathcal{X}'_0 := \mathcal{X}_0 \times \{0\}$ consist of the initial set of states, and the transition function $f' : \mathcal{X}' \rightarrow \mathcal{X}'$ is defined as:

$$(4) \quad f'((x, \ell)) = \begin{cases} (f(x), \ell + 1) & \text{for all } x \in \mathcal{X}_{VF}, \\ (f(x), \ell) & \text{otherwise,} \end{cases}$$

for all $i \in \mathbb{N}$. Observe that any state sequence $\langle x_0, x_1, \dots \rangle$ of the system $\mathfrak{S}$ that visits the region $\mathcal{X}_{VF}$ only finitely often must have some index $i \in \mathbb{N}$, with $x_j \notin \mathcal{X}_{VF}$ for all $j \geq i$. This corresponds to a state sequence $\langle x'_0, x'_1, \dots \rangle$ of the system $\mathfrak{S}'$ such that there exists some $k \in \mathbb{N}$ with $x'_{j+1} = (f(j), k)$ for all $j \geq i$. Therefore showing the system $\mathfrak{S}$ visits $\mathcal{X}_{VF}$ only finitely often reduces to showing the system $\mathfrak{S}'$ never reaches a state $(x, k+1)$ for some $k \in \mathbb{N}$, and some $x \in \mathcal{X}_{VF}$. We now define CBBCs over the original system by taking this modified system's dynamics into account.

Definition 3.1. A function $\mathcal{B} : \mathcal{X} \times [0..k] \rightarrow \mathbb{R}$ is a co-Büchi barrier certificate (CBBC) for a system $\mathfrak{S}$ with set of states $\mathcal{X}_{VF}$ that must be visited only finitely often if there exists some $k \in \mathbb{N}$ such that:

$$(5) \quad \mathcal{B}(x, 0) \leq 0, \text{ for all } x \in \mathcal{X}_0,$$

$$(6) \quad \mathcal{B}(x, k) > 0, \text{ for all } x \in \mathcal{X}_{VF},$$

and for all values $i \in [0..k]$, and $j \in [0..(k-1)]$ we have:

$$(7) \quad \mathcal{B}(f(x), i) - \mathcal{B}(x, i) \leq 0, \text{ for all } x \notin \mathcal{X}_{VF} \text{ and,}$$

$$(8) \quad \mathcal{B}(f(x), j+1) - \mathcal{B}(x, j) \leq 0, \text{ for all } x \in \mathcal{X}_{VF}.$$

Theorem 3.2. Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, with a region $\mathcal{X}_{VF}$. Suppose there exists a $k \in \mathbb{N}$, and a function $\mathcal{B} : \mathcal{X} \times [0..k] \rightarrow \mathbb{R}$ as in Definition 3.1. Then the traces of the system visits the region $\mathcal{X}_{VF}$ at most k times.

Proof. We prove this by contradiction. Suppose there exists a state sequence $\langle x_0, x_1, \dots \rangle$ of system $\mathfrak{S}$ that visits the region $\mathcal{X}_{VF}$ more than k times. In particular, let $t \in \mathbb{N}$ be the index at which the state sequence visits $\mathcal{X}_{VF}$ for the $(k+1)$ -th time. This corresponds to a state sequence $\langle x'_0, x'_1, \dots \rangle$ for system $\mathfrak{S}'$ such that $x_t = (x_t, k)$, and $x'_{t+1} = (x_{t+1}, k+1)$. Following condition (5), observe that we have $\mathcal{B}(x_0, 0) \leq 0$. For each state $x'_i = (x_i, \ell_i)$, either $x_i \in \mathcal{X}_{VF}$, in which case, we have $x'_{i+1} = (x_{i+1}, \ell_i + 1)$, and condition (8) holds, or $x'_{i+1} = (x_{i+1}, \ell_i)$, and condition (7) holds via induction for any $i \in \mathbb{N}$. Thus, we have $\mathcal{B}(x_i, \ell_i) \leq 0$ for all $i \in \mathbb{N}$. Therefore $\mathcal{B}(x_t, k) \leq 0$, but this contradicts with condition (6) which requires $\mathcal{B}(x_i, k+1) > 0$. $\square \quad \square$

Observe that if we set $k = 0$ and consider $\mathcal{X}_u = \mathcal{X}_{VF}$, we recover the original formulation of barrier certificates as in Definition 2.1. Here conditions (5) corresponds to condition (1) over the initial set of states, condition (6) corresponds to condition (2) over $\mathcal{X}_u$, and conditions (7) corresponds to condition (3). We now show how CBBCs are not only sufficient but also necessary with respect to oracles. To do so we define a notion of oracle functions that keep track of whether a state is visited or not, and another oracle function to determine the number of times a state visits $\mathcal{X}_{VF}$. For easier presentation, we first define these functions as follows. Let

oracle function $\mathcal{O} : \mathcal{X} \rightarrow 2^{\mathcal{X}}$ be defined as $\mathcal{O}(x) = \{f^i(x) \mid i \in \mathbb{N}\}$ for all states $x \subseteq \mathcal{X}$. To keep track of number of visits, we define oracle function $\mathcal{O}_{\mathcal{X}_{VF}} : \mathcal{X} \rightarrow \mathbb{N} \cup \{\infty\}$ for any $x \in \mathcal{X}$ as:

- $\mathcal{O}_{\mathcal{X}_{VF}}(x) := \infty$, if, for all $i \in \mathbb{N}$, there exists $j \in \mathbb{N}_{\geq i}$ such that $f^j(x) \in \mathcal{X}_{VF}$.
- $\mathcal{O}_{\mathcal{X}_{VF}}(x) := j$ if there are at most j -indices $i_1, \dots, i_j$ such that $f^{i_k}(x) \in \mathcal{X}_{VF}$, and for all indices $\kappa \notin \mathbb{N} \setminus \{i_1, \dots, i_j\}$, we have $f^\kappa(x) \notin \mathcal{X}_{VF}$. Note that $i_1 = 0$ is a valid index. That is, if $x \in \mathcal{X}_{VF}$, we have $\mathcal{O}_{\mathcal{X}_{VF}}(x_0) \in \mathbb{N}_{\geq 1} \cup \{\infty\}$ as $f^0(x) = x$.
- $\mathcal{O}_{\mathcal{X}_{VF}}(x) := 0$ if for all $i \in \mathbb{N}$, we have $f^i(x) \notin \mathcal{X}_{VF}$.

Intuitively, the above oracle functions $\mathcal{O}$ and $\mathcal{O}_{\mathcal{X}_{VF}}$ determine if some state is reachable, as well as the maximum number of times that the state x visits the set $\mathcal{X}_{VF}$. observe that the set $Reach := \bigcup_{x_0 \in \mathcal{X}_0} \mathcal{O}(x_0)$ is

the set of states that are reachable from the initial state set while $\mathcal{O}_{\mathcal{X}_{VF}}(x)$ for any $x \in Reach$ denotes the number of times x visits the set $\mathcal{X}_{VF}$.

Lemma 3.3. *Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ with a set of states $\mathcal{X}_{VF}$ that is visited only finitely often. Then there exists $k \in \mathbb{N}$, and a CBBC $\mathcal{B} : \mathcal{X} \times [0..k] \rightarrow \mathbb{R}$ as in Definition 3.1.*

Proof. First, we bound the maximum number of visits to $\mathcal{X}_{VF}$ to determine the value of $k \in \mathbb{N}$ as follows: Let $k = \max\{j \mid j := \mathcal{O}_{\mathcal{X}_{VF}}(x) \text{ and } x \in Reach\}$ denote the maximum number of times some state $x \in \mathcal{X}_{VF}$ may be reached from the initial set of states. Observe that there cannot exist $k' \in \mathbb{N}_{>k}$ such that $k' = \mathcal{O}_{\mathcal{X}_{VF}}(x)$ for some $x \in Reach$ and so k provides the maximum bound on the number of times a state $\mathcal{X}_{VF}$ may be reached. We now define the CBBC $\mathcal{B} : \mathcal{X} \times [0..k] \rightarrow \mathbb{R}$ as:

$$\mathcal{B}(x, i) = \begin{cases} 0 & \text{if } x \in Reach, \text{ and } i + \mathcal{O}_{\mathcal{X}_{VF}}(x) \leq k, \\ 1 & \text{otherwise.} \end{cases}$$

We now show that $\mathcal{B}$ is a CBBC. Observe that condition (5) holds for any state $x_0 \in \mathcal{X}_0$, as $0 + \mathcal{O}_{\mathcal{X}_{VF}}(x_0) \leq k$ from our assumption that k is the maximum number of times a state in $\mathcal{X}_{VF}$ may be reached. For condition (6), let us assume that $\mathcal{B}(x_{vf}, k) \leq 0$ for some $x_{vf} \in \mathcal{X}_{VF}$. Then, we must have $k + \mathcal{O}_{\mathcal{X}_{VF}}(x_{vf}) \leq k$. However, we have $\mathcal{O}_{\mathcal{X}_{VF}}(x_{vf}) \in \mathbb{N}_{\geq 1} \cup \{\infty\}$, and so we have $k + \mathcal{O}_{\mathcal{X}_{VF}}(x_{vf}) \leq k + 1 \leq k$. This is a contradiction, and so condition (6) must also hold. For condition (7) to fail, it must be the case that $\mathcal{B}(x, i) = 0$ for some $x \in \mathcal{X}_{VF}$, and $\mathcal{B}(f(x), i+1) = 1$. If $\mathcal{B}(f(x), i+1) = 1$, then either $f(x) \notin Reach$, or $i+1 + \mathcal{O}_{\mathcal{X}_{VF}}(f(x_{vf})) > k$. However, we cannot have $x \in Reach$, and $f(x) \notin Reach$ by definition of $Reach$. The only other option is $i+1 + \mathcal{O}_{\mathcal{X}_{VF}}(f(x)) > k$. However, we know that $i + \mathcal{O}_{\mathcal{X}_{VF}}(x) \leq k$ as $\mathcal{B}(x, i) = 0$, and furthermore $\mathcal{O}_{\mathcal{X}_{VF}}(x) \in \mathbb{N}_{\geq 1}$, as $x = f^0(x)$. Thus $\mathcal{O}_{\mathcal{X}_{VF}}(x) = t+1$ for some $t \in \mathbb{N}$, and so we must have $i+t+1 \leq k$. Observe that if this is true, there are at most $t+1$ indices such that $f^{i_k}(x) \in \mathcal{X}_{VF}$ for some $i_1, i_2, \dots, i_{t+1} \in \mathbb{N}$, and in particular, we must have $i_1 = 0$ as $x \in \mathcal{X}_{VF}$. Furthermore, $f^\ell(x) \notin \mathcal{X}_{VF}$ for any $\ell \notin \{0, i_2, \dots, i_{t+1}\}$. Thus, $f(x)$, has only t indices $\{(i_2 - 1), \dots, (i_{t+1} - 1)\}$ such that $f^{i_k-1}(f(x)) \in \mathcal{X}_{VF}$, and so we must have $\mathcal{O}_{\mathcal{X}_{VF}}(f(x)) = t$. This is a contradiction as we cannot have $i+1+t > k$ from $\mathcal{B}(f(x), i) = 1$, and $i+t+1 \leq k$ from $\mathcal{B}(x, i) = 0$. A similar contradiction arises for condition (8), however the values of $\mathcal{O}_{\mathcal{X}_{VF}}(x)$, and $\mathcal{O}_{\mathcal{X}_{VF}}(f(x))$ are the same if $x \notin \mathcal{X}_{VF}$. $\square$ $\square$

4. VERIFYING AUTOMATA PROPERTIES

We now extend the results in Section 3 to verify properties specified by an automaton. Let $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ be a dynamical system, Σ be a finite alphabet and $\mathcal{L} : \mathcal{X} \rightarrow \Sigma$ be a labeling function. Consider $\mathcal{A}_{c,k} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ to be a k -UCA automaton representing a specification. Then the goal of verification is to show that $TR(\mathfrak{S}, \mathcal{L}) \subseteq L(\mathcal{A}_{c,k})$. Observe that any trace that is accepted by the automaton visits an accepting state at most k -times. This provides our approach for verifying dynamical systems against properties specified by automata.

We first take a synchronous product between the system $\mathfrak{S}$ and automaton $\mathcal{A}_{c,k}$. As the state set of the automaton is finite, we can equivalently represent the states of the automaton as natural numbers in the

set $\{0, 1, \dots, (|Q| - 1)\}$, represented succinctly as $[0..(|Q| - 1)]$. We define the product as a tuple $\hat{\mathfrak{S}} = (\hat{\mathcal{X}}, \hat{\mathcal{X}}_0, \hat{f}, \hat{\mathcal{X}}_{Acc})$, where $\hat{\mathcal{X}} = X \times [0..(|Q| - 1)]$ denotes the state set, the initial set of states is defined as $\hat{\mathcal{X}}_0 = \mathcal{X}_0 \times \{i \mid i \in [0..(|Q| - 1) \cap Q_0]\}$. Observe that while the dynamical system has a unique successor state for any state $x \in \mathcal{X}$, the automata may be nondeterministic and so the state transition map is not a function but rather a relation for the product that is also nondeterministic. The state transition relation $\hat{f} \subseteq \hat{\mathcal{X}} \times \hat{\mathcal{X}}$ is defined as follows for all $x \in \mathcal{X}$ and $i, j \in [0..(|Q| - 1)]$, as $\hat{f}((x, i)) = \{(f(x), j) \mid j \in \delta(i, \mathcal{L}(x))\}$. Given elements $x \in \mathcal{X}$, and $i \in [0..(|Q| - 1)]$, the set $\hat{f}((x, i))$ has at most $(|Q| - 1)$ elements as $j \in [0..(|Q| - 1)]$. For convenience we use $\hat{\mathcal{X}}_{Acc} = \mathcal{X} \times \{\ell \mid \ell \in Q_{Acc}\}$ to denote the set of accepting states of this product.

A state sequence $\langle x_0, x_1 \dots \rangle$ of system $\mathfrak{S}$ is accepted by the automaton $\mathcal{A}_{c,k}$ if every corresponding state sequence $\langle \hat{x}_0, \hat{x}_1, \dots \rangle$ has at most k -states in $\hat{\mathcal{X}}_{Acc}$, *i.e.*, there exists $i_1, i_2, \dots, i_k \in \mathbb{N}$, with $\hat{x}_{i_j} \in \hat{\mathcal{X}}_{Acc}$, and for all $\ell \notin \{i_1, i_2, \dots, i_k\}$, we have $\hat{x}_\ell \notin \hat{\mathcal{X}}_{Acc}$. Our approach to verify whether a system satisfies the specification is to determine whether all traces of the product $\hat{\mathfrak{S}}$ visits states in the set $\hat{\mathcal{X}}_{Acc}$ at most k times. If so then the system $\mathfrak{S}$ satisfies the specification, and if not our approach is inconclusive. To determine this we make use of a CBBC on the product system.

To determine this we make use of a CBBC on the product system defined as follows.

Definition 4.1. A function $\mathcal{B} : \mathcal{X} \times Q \times [0..k] \rightarrow \mathbb{R}$ is a co-Büchi barrier certificate for the synchronous product $\hat{\mathfrak{S}}$ of a system $\mathfrak{S}$ and a k -UCA $\mathcal{A}_{c,k}$ if:

$$(9) \quad \mathcal{B}(x, \ell, 0) \leq 0, \quad \text{for all } x \in \mathcal{X}_0, \ell \in Q_0 \setminus Q_{Acc},$$

$$(10) \quad \mathcal{B}(x, \ell, k) > 0, \quad \text{for all } x \in \mathcal{X}, \ell \in Q_{Acc},$$

and for all states $x \in \mathcal{X}$, counter values $i \in [0..k]$, $j \in [0..(k - 1)]$, and all states $\ell \in [0..(|Q| - 1)]$, and all states $\ell' \in \delta(\ell, \mathcal{L}(x))$, we have

$$(11) \quad \max_{\ell'} \left\{ \mathcal{B}(f(x), \ell', i) \right\} \leq \mathcal{B}(x, \ell, i), \quad \text{if } \ell \notin Q_{Acc},$$

$$(12) \quad \max_{\ell'} \left\{ \mathcal{B}(f(x), \ell', j + 1) \right\} \leq \mathcal{B}(x, \ell, j), \quad \text{if } \ell' \in Q_{Acc}.$$

Theorem 4.2. Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a k -UCA $\mathcal{A}_{c,k} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$, and a labeling map $\mathcal{L} : \mathcal{X} \rightarrow \Sigma$. Suppose there exists $k \in \mathbb{N}$, and a function $\mathcal{B} : \mathcal{X} \times Q \times [0..k]$ as in Definition 4.1, then $TR(\mathfrak{S}, \mathcal{L}) \subseteq L(\mathcal{A}_{c,k})$.

Proof. We observe that the function $\mathcal{B} : \mathcal{X} \times Q \times [0..k] \rightarrow \mathbb{R}$ satisfying conditions (9) to (12) is a CBBC as in Definition 3.1 for the product $\hat{\mathfrak{S}}$ where the set of states that must be visited only finitely often $\hat{\mathcal{X}}_{VF}$ is $\hat{\mathcal{X}}_{Acc}$. Following Theorem 3.2, we note that every state sequence of of system $\hat{\mathfrak{S}}$ must visit the region $\hat{\mathcal{X}}_{Acc}$ at most k -times, therefore every trace of $\mathfrak{S}$ under labeling map must be in the language of the automaton $\mathcal{A}_{c,k}$. $\square$ $\square$

To verify whether the traces of $\mathfrak{S}$ under a labeling function $\mathcal{L}$ is accepted by a UCA $\mathcal{A}_c = (\Sigma, Q, Q_0, \delta, Q_{Acc})$, we search for a suitable k such that the traces of $\mathfrak{S}$ are in the language of the k -UCA $\mathcal{A}_{c,k} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ following Corollary 2.5. This is then cast as searching for a suitable CBBC for the product. If one is successful in finding such a CBBC for some $k \in \mathbb{N}$, then we infer the traces of the system to be in the language of the automaton. Here, the value k is initially set to 0, and is increased incrementally by one, every time we fail to find a CBBC until a suitable threshold is reached. If a CBBC is found before the threshold is reached we stop our search and guarantee that the traces of the system are in the language of $\mathcal{A}_c$ and if not our procedure is inconclusive. Note that one *does not need* to take the product of the system and the automaton explicitly to check the satisfaction of conditions and can define functions that are piecewise over automata states similar to [23]. To verify whether the traces of a system $\mathfrak{S}$ under a labeling function $\mathcal{L}$ are not contained in the language of an NBA $\mathcal{A}_b = (\Sigma, Q, Q_0, \delta, Q_{Acc})$, We first determine whether the traces of the system $\mathfrak{S}$ are contained in

the language of the UCA $\mathcal{A}_c = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ as specified in the previous paragraph. If so, then no trace of the system is in the language of the NBA $\mathcal{A}_b$ and if not our procedure is inconclusive.

4.1. Limitations of CBBCs. While, we have shown that CBBCs are not only sufficient, but also necessary conditions to verify k -UCA properties they unfortunately, do not lend themselves readily to synthesis. The key challenge for synthesis of controllers against ω -regular specifications is that one may not use nondeterministic or universal automata due to issues related to realizability stemming from the nondeterminism or universality [29, 30]. Thus, one must instead use deterministic variants of these automata, however deterministic Büchi automata cannot capture all ω -regular specifications. Instead one must use deterministic ω -automata such as parity or rabin automata [8]. However, the accepting conditions of these automata need to show not only that a region is visited finitely often but also that some regions are visited infinitely often. A direct application of CBBCs to show regions are visited infinitely often, leads to one having to deal with unbounded counters making them a poor choice for automated approaches. However, CBBCs may provide an effective tool to synthesize controllers for classes of ω -automata such as as good-for-games automata [31]. We leave an investigation of this as future work.

5. CBBC COMPUTATION

We present two approaches to compute co-Büchi barrier certificates as in Definition 3.1. The first relies on using Satisfiability Modulo Theory (SMT) solvers, while the second is based on a sum-of-squares (SOS) approach. To find a CBBC as in Definition 3.1, we first fix the template of the CBBC to be a piece-wise polynomials of a fixed degree d with unknown coefficients $c_1, \dots, c_z$, i.e. $\mathcal{B}_i(x) = \sum_{m=1}^z c_{m,i} p_{m,i}(x)$ where the functions p_m are monomials over the states x and function $\mathcal{B}_i$ are defined for each counter value $i \in [0..k]$. As $i \in [0..k]$ takes only finitely many values, we consider $\mathcal{B}$ to be a piecewise function with k pieces. We now discuss how we make use of a sum-of-squares approach to search for CBBCs.

5.1. Sum-of-Squares based Approach. One may make use of a semidefinite programming approach [15] via sum-of-squares (SOS) similar to a search for a standard barrier certificates [10]. A set $A \subseteq \mathbb{R}^n$ is semi-algebraic if it can be defined with the help of a vector of polynomial inequalities $h(x)$ as $A = \{x \mid h(x) \geq 0\}$, where the inequalities is interpreted component-wise.

To adopt a SOS approach to find CBBCs as in Definition 3.1, we consider the sets $\mathcal{X}$, $\mathcal{X}_0$, and $\mathcal{X}_{VF}$ to be semi-algebraic sets defined with the help of vectors of polynomial inequalities g , g_0 , and g_{VF} respectively. As these sets are semi-algebraic, the set $\mathcal{X} \setminus \mathcal{X}_{VF}$ is semi-algebraic as well, and let this set be defined by polynomial inequalities g_N . Furthermore, we assume that the function f is polynomial. Then the search for a CBBC reduces to showing that the following polynomials are sum-of-squares for all $i \in [0..k]$, and $j \in [0..(k-1)]$:

$$(13) \quad -\mathcal{B}_0(x) - \lambda_0^T(x) g_0(x),$$

$$(14) \quad \mathcal{B}_k(x) - \lambda_{VF}^T(x) g_{VF}(x) - \varepsilon$$

$$(15) \quad -\mathcal{B}_i(f(x)) + \mathcal{B}_i(x) - \lambda_{N,i}^T(x) g_N(x), \text{ and}$$

$$(16) \quad -\mathcal{B}_{(j+1)}(f(x)) + \mathcal{B}_{(j)}(x) - \lambda_{V,i}^T(x) g_{VF}(x),$$

where the multipliers λ_0 , λ_{VF} , $\lambda_{N,i}$, and $\lambda_{V,i}$ are sum-of-squares over $\mathcal{X}$, and $\varepsilon \in \mathbb{R}_{>0}$ is some small positive value.

Lemma 5.1. *Assume the sets $\mathcal{X}$, $\mathcal{X}_0$, and $\mathcal{X}_{VF}$ are semi-algebraic, and there exists k sum-of-squares polynomials $\mathcal{B}_\ell(x)$ satisfying conditions (13) to (16) for all $0 \leq \ell \leq k$. Then the function $\mathcal{B}(x, i) = \mathcal{B}_i(x)$ is a CBBC satisfying conditions (5) to (8).*

To find an SOS formulation for the product, first observe that there are finitely many letters $\sigma \in \Sigma$, and so without loss of generality, one can partition the set $\mathcal{X}$ into finitely many partitions $\mathcal{X}_{\sigma_1}, \dots, \mathcal{X}_{\sigma_p}$ where for all $x \in \mathcal{X}_{\sigma_m}$, we have $\mathcal{L}(x) = \sigma_m$. Given an element $\sigma_m \in \Sigma$, we can uniquely characterize the relation δ_{σ_m} as $(\ell, \ell') \in \delta_{\sigma_m}$ if and only if $\ell' \in \delta(\ell, \sigma_m)$. Assume that the sets $\mathcal{X}$, $\mathcal{X}_0$, and $\mathcal{X}_{\sigma_m}$ for all σ_m are semi-algebraic and characterized by polynomial vector of inequalities $g(x)$, $g_0(x)$, and $g_{\sigma_m}(x)$, respectively. Furthermore, assume that the state transition function $f : \mathcal{X} \rightarrow \mathcal{X}$ is polynomial. Then similar to the conditions above, one can reduce a search for CBBCs to checking if relevant polynomials are SOS. To do so, we assume that the CBBC $\mathcal{B}(x, \ell, i)$ is defined as piecewise functions $\mathcal{B}_{\ell, i}(x)$ for each state of the automaton $\ell \in Q$, as there are finitely many states in Q , and similarly, for a fixed k , i , and j take on value from $[0..k]$ and $[0..(k-1)]$ respectively. Now, one can reduce the search for a CBBC to showing that the following polynomials are SOS:

$$(17) \quad -\mathcal{B}_{\ell, 0}(x) - \lambda_0^T(x)g_0(x), \text{ for all } \ell \in Q_0,$$

$$(18) \quad \mathcal{B}_{\ell, k}(x) - \lambda_1^T(x)g(x) - \varepsilon, \text{ for all } \ell \in Q_{Acc},$$

where $\varepsilon \in \mathbb{R}_{>0}$ is some small positive value, and for all $\sigma_m \in \Sigma$, $i \in [0..k]$ and all $\ell \in Q \setminus Q_{Acc}$, $\ell' \in \delta(\ell, \sigma_m)$ the following polynomial is sum-of-squares:

$$(19) \quad -\mathcal{B}_{\ell, i}(x) + \mathcal{B}_{\ell', i}(f(x)) - \lambda_{2, \sigma_m, \ell, i}^T(x)g_{\sigma_m}(x),$$

and for all $\sigma_m \in \Sigma$, $j \in [0..(k-1)]$ and all $\ell \in Q \cap Q_{Acc}$, $\ell' \in \delta(\ell, \sigma_m)$ the following polynomial is sum-of-squares:

$$(20) \quad -\mathcal{B}_{\ell, j}(x) + \mathcal{B}_{\ell, (j+1)}(f(x)) - \lambda_{3, \sigma_m, \ell, i}^T(x)g_{\sigma_m}(x),$$

where the multipliers $\lambda_0, \lambda_1, \lambda_{2, \sigma_m, \ell, i}, \lambda_{3, \sigma_m, \ell, i}$ are all sum-of-squares polynomials over the states $\mathcal{X}$. Note we enumerate the conditions for every $\sigma_m \in \Sigma$ and explicitly specify the values ℓ' based on the relation δ_{σ_m} .

Lemma 5.2. *Assume the sets $\mathcal{X}$, $\mathcal{X}_0$, $\mathcal{X}_{VF}$, and $\mathcal{X}_{\sigma_i}$ for all σ_i are semi-algebraic, and there exists sum-of-squares polynomials $\mathcal{B}_{\xi, \ell}(x)$ satisfying conditions (17) to (20) for all $\xi \in Q$, and $\ell \in [0..k]$. Then the function $\mathcal{B}(x, \xi, \ell) = \mathcal{B}_{\xi, \ell}(x)$ is a CBBC for the product satisfying conditions (9) to (12).*

One can then encode conditions (13) to (16) as well as conditions (17) to (20) in a suitable SOS solver such as TSSOS [32] by fixing the barrier certificate to be a polynomial of degree d and checking if the above constraints are satisfiable.

The time complexity of checking if a polynomial in n variables of degree d is SOS involves solving a semidefinite feasibility problem in $\binom{n+d}{d}$ variables [15]. For the above approach, we have $O(k)$ polynomial constraints, as we can define function $\mathcal{B}$ to be a set of k -functions that are piecewise over the values of k . Thus, computing a CBBC is polynomial in $O(k \binom{n+d}{d})$. When dealing with the product of the system and a k -UCA $\mathcal{A} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$, this is now polynomial in $O(k|Q| \binom{n+d}{d})$ variables. We summarize the Algorithm for the SOS-based approach to verify UCA specifications in Algorithm 1.

5.2. SMT-based Approach. Alternative to SOS, one may make use of a Counterexample-guided Inductive Synthesis (CEGIS) [33] approach to find these CBBCs. Observe that if the value of $x \in \mathcal{X}$ and $i \in \mathbb{N}$ is fixed, then the only decision variables in $\mathcal{B}(x, i)$ are the coefficients c_m . We sample N points from the state space of the system to create the set $S = \{x_1, \dots, x_N\}$, and encode the constraints of the barrier certificate for each point $x_j \in S$ and all $0 \leq i \leq k$ as an SMT-query over the theory of linear real arithmetic (LRA) [34] using z3 [35]. We then find values $c_1, \dots, c_z$ for the coefficients and substitute them as a candidate CBBC $\mathcal{B}(x, i)$. To determine if this candidate is a CBBC, we now try to find a $x \in \mathcal{X}$ such that one of the conditions (5) to (8) does not hold. We do this by encoding the negation of these conditions as an SMT query. If such a counterexample $x_{counter}$ is found, we add it to the set S and repeat the process. If no such counterexample is found, then we conclude that this function is indeed a CBBC. We adopt a similar approach to find a CBBC for the synchronized product, that acts as a proof that the traces of the system are contained in the language of a k -UCA $\mathcal{A}_{c, k} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$. Formally, we sample N points from the state set $\mathcal{X}$, and check the

Algorithm 1 CBBC computation for verifying k -UCA specifications

```

procedure CBBC_COMP( $\mathfrak{S}, \mathcal{A}_c, \mathcal{L}, k_{\max}, d$ )
  Inputs: System  $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ ,
            UCA  $\mathcal{A}_c = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ ,
            Labeling map  $\mathcal{L} : \mathcal{X} \rightarrow \Sigma$ ,
            Maximum counter threshold  $k_{\max}$ ,
            Degree of the certificates  $d$ .

   $k := 0$ 
  while  $k \leq k_{\max}$  do
    Encode SOS polynomials from conditions (17)-(20) as an SDP following [10].
    if SDP is feasible then
      return ( $status, \{\mathcal{B}_{i,\ell} \mid i \in Q, \ell \in [0..k]\}$ )
    else
       $k := k + 1$ 
    end if
  end while
  return inconclusive
end procedure

```

conjunction of the following queries for every $\ell \in Q$:

$$(21) \quad \bigwedge_{x_j \in \mathcal{X}_0, \ell \in Q_0} \left(\mathcal{B}_{\ell,0}(x_j) \leq 0 \right),$$

$$(22) \quad \bigwedge_{x_j \in \mathcal{X}, \ell \in Q_{Acc}} \left(\mathcal{B}_{\ell,k}(x_j) > 0 \right),$$

and for all states $\ell \in [0..(|Q| - 1)]$ such that $\ell' \in \delta(q_\ell, \mathcal{L}(x_i))$, and for all counter values $i \in [0..k]$, we have

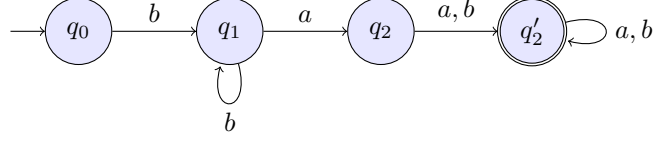
$$(23) \quad \bigwedge_{x_j \in \mathcal{X}_0, \ell \in (Q \setminus Q_{Acc})} \left(\mathcal{B}_{\ell',i}(f(x_j)) \leq \mathcal{B}_{\ell,i}(x_j) \right),$$

$$(24) \quad \bigwedge_{x_j \in \mathcal{X}_0, \ell \in Q_{Acc}} \left(\mathcal{B}_{\ell', (i+1)}(f(x_j)) \leq \mathcal{B}_{\ell,i}(x_j) \right).$$

Unlike SOS, the above approaches are NP-hard [36], and there is no guarantee that our CEGIS approach will terminate. We have shown the usefulness of CBBCs in Theorem 3.2 and Lemma 3.3. Our first question was to investigate whether classical approaches for computing barrier certificates can be used for CBBCs. We now show that our approach generalizes the state triplet approaches presented in [16]. Intuitively, our goal is as follows: we wish to show that if one is able to find a proof that a system $\mathfrak{S}$ satisfies a desired ω -regular property via NBA $\mathcal{A}_b$ through the state-triplet approach, then one is also able to find a proof that $\mathfrak{S}$ satisfies the same property via CBBCs.

6. SUBSUMING THE STATE TRIPLET APPROACH

Our result here follows from a simple observation that there exist systems which cannot be proved to verify UCA properties via the state-triplet approach, however as CBBCs are not only sufficient but also necessary, one can find such certificates for these systems. Furthermore, while the above summary is non-constructive, we provide a constructive proof in Appendix A.1, which shows that the degree of our CBBCs are at most the same degree as the barrier certificates obtained from the triplet approach. A similar discussion of such proofs also appears in [23] in the context of closure certificates and transition invariants rather than barrier certificates.

FIGURE 2. Automaton $\mathcal{A}'_{b_1}$ created from unfolding NBA $\mathcal{A}_b$ in Section 1 (Figure 1).

6.1. The State Triplet Approach. Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, an NBA $\mathcal{A}_b = (\Sigma, Q, Q_0, \delta, Q_{Acc})$, and a labeling function $\mathcal{L} : \mathcal{X} \rightarrow \Sigma$. We first unroll the simple cycles of the automaton $\mathcal{A}_b$ that start from an accepting state to construct an automaton $\mathcal{A}'_b = (\Sigma, Q', Q_0, \delta', Q'_{Acc})$ such that $L(\mathcal{A}'_b) = L(\mathcal{A}_b)$. The states Q' of $\mathcal{A}'_b$ contain all the states Q of $\mathcal{A}_b$ as well as one new state q'_i for every state $q_i \in Q$ that can be reached from an accepting state. Determining which states are reachable from an accepting state can be done in polynomial time [37] in the number of states of the automaton. Intuitively, these new states are meant to determine the state runs of the system which have reached an accepting state and continue onward. The set of initial states is the same as $\mathcal{A}_b$, while the set of accepting states consist of the states q'_{Acc} , *i.e.*, those accepting states that can be reached from an accepting state. The transitions δ' are the same as the transitions for $\mathcal{A}_b$, except for the transitions that leave an accepting state $q_{Acc} \in Q_{Acc}$ and additional transitions added. The transitions added are due to two reasons :

- For every pair of states $q_i, q_j \in Q$ that are reachable from an accepting state, and for any $\sigma \in \Sigma$ such that $q_j \in \delta(q_i, \sigma)$, we add a transition $q'_j \in \delta(q'_i, \sigma)$. This preserves all the transitions for the newly added states so they behave the same as before.
- For every state $q_i \in Q$, and every accepting state $q_{Acc} \in Q_{Acc}$, and any letter $\sigma \in \Sigma$ such that $q_j \in \delta(q_{Acc}, \sigma)$, we add a transition $q'_j \in \delta(q_{Acc}, \sigma)$. For every edge that leaves the accepting state, we add an edge to move to the states of the form $q' \in Q' \setminus Q$. Observe that there are no transition from the states in $Q' \setminus Q$ to the ones in Q .

To ensure that $T(\mathfrak{S}, \mathcal{L}) \cap L(\mathcal{A}_b) = \emptyset$, it is sufficient to ensure that there exists a barrier certificate between some edge pair (or a state triplet) for every simple path from an initial state $q_0 \in Q_0$ to some accepting state $q'_{Acc} \in Q'_{Acc}$ in the automaton $\mathcal{A}'_b$. Note that unrolling the cycles more than once does not change the edge pairs that enter or exit a state, and thus does not change the labels of the state triplets (edge pairs) that are encountered along a simple path from an initial state to an accepting state. One may perform the same construction on the automaton again, however this does not add any new state triplets to consider. The proof for this appears in [23, Lemma 9]. Thus, one is unable to verify those traces of a system which reach and cycle on an accepting state more than twice, even if they visit an accepting state only finitely often. Note that unrolling once does have an impact since we can now consider those state triplets along the simple paths from an accepting state to an accepting state.

To illustrate the state triplet approach, consider the NBA $\mathcal{A}_{b_1} = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ in Figure 1, discussed in the introduction, with $\Sigma = \{a, b\}$, $Q = \{q_0, q_1, q_2\}$, $Q_0 = \{q_0\}$, $Q_{Acc} = \{q_2\}$ and δ specified by the edges in the graph. This NBA accepts those words which start with b and have at least one a in them and can be used to verify the safety of the system. We, unfold $\mathcal{A}_{b_1}$ to get the NBA $\mathcal{A}'_{b_1} = (\Sigma', Q', Q_0, \delta, Q'_{Acc})$ as shown in Figure 2.

Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, under a labeling map $\mathcal{L}$ which naturally partitions $\mathcal{X}$ into two sets $\mathcal{X}_a$ and $\mathcal{X}_b$. We break the simple paths of the above automaton into state triplets or edge pairs and then find barrier certificates over these edge pairs. In this case there is a single simple path from the initial state q_0 to an accepting state q'_2 consisting of the states (q_0, q_1, q_2, q'_2) and leads to two triplets (q_0, q_1, q_2) and (q_1, q_2, q'_2) . We take the first triplet and note this corresponds to the edges with labels a and b . We then search for a barrier certificate by considering the initial set of states to be the set $\mathcal{X}_b$, the set of states with a label b , and the unsafe set of states $\mathcal{X}_u$ to be the set $\mathcal{X}_a$, all the states with a label a . If we are successful in finding such a barrier certificate, then we infer that we may cut this path and observe that no other path from the initial

state q_0 to the accepting state q'_2 exists. In such a case we verify that $T(\mathfrak{S}, \mathcal{L}) \cap L(\mathcal{A}_b) = \emptyset$ and so that no trace of the system is in the language of NBA $\mathcal{A}_b$. If we cannot find such a barrier certificate, we now search for a barrier certificate in the next triplet, the triplet (q_0, q_1, q_2) . Observe that the edges (q_1, q_2) and (q_2, q'_2) share the label a , and as such one cannot simultaneously satisfy condition (1) and (2) on the states with the label a . When this fails, our approach is inconclusive and we can say nothing about whether the system satisfies the property or not.

The state triplet approach is conservative in the following direction: Independent of the state runs in the automaton and of the initial states of the system $\mathfrak{S}$, one is required to break the edge pairs of every simple path regardless of what states of the automaton may be encountered before or after.

Theorem 6.1. *Let $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ be a system, $\mathcal{L} : \mathcal{X} \rightarrow \Sigma$ be a labeling map, and $\mathcal{A}_b = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ be an NBA. Suppose there exist barrier certificates $\mathcal{B}_1, \dots, \mathcal{B}_m$ that act as a proof that no trace of the system is in $L(\mathcal{A}_b)$ by the state triplet approach, then there exists a CBBC $\mathcal{B}$ that also serves as a witness that no trace is in $L(\mathcal{A}_b)$.*

We present a constructive proof of Theorem 6.1 in Appendix A.1. A nonconstructive proof follows from the fact that there exist systems which cannot be verified via the state triplet approach but can be via CBBCs.

7. CASE STUDIES

The first case study demonstrates that while the state triplet approach is inadequate in demonstrating the satisfaction of the property, our approach finds CBBCs to verify the satisfaction of the property. We then use the SOS-based approach to find certificates for higher dimensional oscillator models to demonstrate their effectiveness.

7.1. Case Study Against the State Triplet Approach. Consider a room-temperature model adapted from [38], whose dynamics is specified as:

$$\mathfrak{S} : x(t+1) = (1 - \tau_s \alpha)x(t) + \tau_s \alpha T_e,$$

To demonstrate the effectiveness of our approach against properties specified by automata, we consider the same system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ as in the previous Subsection, with $\mathcal{X} = [17, 40]$ as the state set, $\mathcal{X}_0 = [30, 35]$ as the initial set of states, and $f(x) = (1 - \tau_s \alpha)x + \tau_s \alpha T_e$ denotes the state transition function, where $\tau_s = 5$ minutes denotes the sampling time, $\alpha = 0.08$ the heat-exchange coefficient and $T_e = 17^\circ\text{C}$ denotes the ambient temperature. Consider the UCA $\mathcal{A}_c = (\Sigma, Q, Q_0, \delta, Q_{Acc})$ specified in Figure 3, where $Q = \{q_0, q_1\}$ denotes the set of states, $Q_0 = \{q_0\}$ the initial state, $Q_{Acc} = \{q_1\}$ the accept state, and the transition function δ specified by the edges in the graph. Let the labeling function $\mathcal{L} : \mathcal{X} \rightarrow \Sigma$ be defined as:

$$\begin{aligned} \mathcal{L}(x) &= a, & \text{for all } x \in (28, 40], \\ \mathcal{L}(x) &= b, & \text{for all } x \in [25, 28], \text{ and} \\ \mathcal{L}(x) &= c & \text{for all } x \in [17, 25). \end{aligned}$$

First, we observe that we cannot make use of the state triplet approach. The initial set of states have a label a and can reach states with a label b , thus one cannot cut the edge pair (q_0, q_0) and (q_0, q_1) . Similarly, one

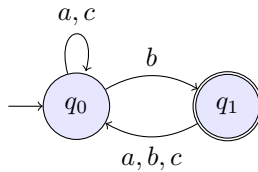


FIGURE 3. UCA $\mathcal{A}_c$ for Subsection 7.1.

cannot cut the edge pair from q_1 to q_0 as the edge pairs (q_0, q_1) and (q_1, q_0) as well as the pair (q_1, q_0) , and (q_0, q_0) have overlapping labels. We thus make use of CBBCs to guarantee that the accepting state q_1 is visited finitely often.

We fix the value of k to be 4, and consider a degree-3 polynomial $\mathcal{B}$ in x (i.e. the state of the system), i (i.e. the state of the automaton), and ℓ (i.e. the counter value). We then adopt a CEGIS approach to find such a CBBC, and found the function $\mathcal{B}(x, i, \ell) = -0.000311\ell^3 - 0.01770x\ell + 0.0727\ell^2 + 0.0442x\ell + 0.30096i\ell - \ell - 0.001$. The time taken to find this CBBC was 1.5 hours on the reference machine.

7.2. Verification for a 2D Van der Pol oscillator model. Next, we consider a 2D van-der Pol Oscillator, whose model dynamics is given by the following difference equations:

$$\mathfrak{S} \begin{cases} x(t+1) = x(t) + Ty(t), \\ y(t+1) = y(t) + T(-x(t) + uy(t)(1 - x(t)^2)), \end{cases}$$

where $T = 0.1$ is the sampling time and $u = 0.4$. The state set, and initial set, are given by $\mathcal{X} = [-2.5, 4] \times [-2.5, 2.5]$, $\mathcal{X}_0 = [3, 3.5] \times [1.5, 2]$ respectively. We consider the UCA $\mathcal{A}_c$ in Figure 4, with

$$\mathcal{L}(x, y) = \begin{cases} a & \text{if } x \in [3, 4], y \in [-1, 0.5], \\ c & \text{if } x \in [2, 4], y \in [-2.5, -2], \\ b & \text{otherwise.} \end{cases}$$

We then search for a CBBC as in Definition 4.1 following Algorithm 1 by using the TSSOS [32] package by setting $k_{\max} = 8$, and incrementing the value of d from $d = 1$ to $d = 8$. We are able to find a CBBC of degree 6 with $k = 3$, ensuring that the accepting states q_1 , and q_2 are visited at most 3 times from an initial state of the system. The time taken for this is around 5 minutes on the reference machine and the certificates are described in Appendix B.

7.3. Verification of a 3D Kuramoto Oscillator Model. As a final case study, we consider a three dimensional Kuramoto oscillator where the dynamics are given by:

$$\mathfrak{S} \begin{cases} x(t+1) = x(t) + TK \sin(y(t) - x(t)) \\ \quad + cx(t)^2 + \Omega, \\ y(t+1) = y(t) + TK (\sin(x(t) - y(t)) \\ \quad + \sin(z(t) - y(t))) + \Omega + cy(t)^2, \\ z(t+1) = z(t) + TK \sin(y(t) - z(t)) \\ \quad + cz(t)^2 + \Omega, \end{cases}$$

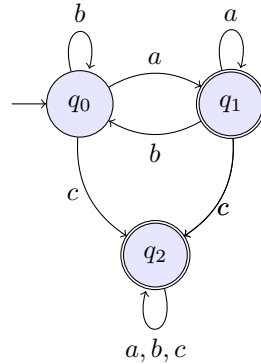


FIGURE 4. UCA $\mathcal{A}_c$ for Subsection 7.2.

where $T = 0.1s$ is the sampling time, $\Omega = 1$ is the natural frequency added to an offset, $K = 0.06$ is the coupling strength, and $c = -0.532$. As the above dynamics uses a trigonometric function, we approximate it using a Taylor's approximation as $\sin(r) \approx r - \frac{r^3}{6}$ for all the corresponding sine terms. We consider $\mathcal{X} = [0, 2]^3$ to be the set of states, $\mathcal{X}_0 = [0, \frac{2\pi}{15}]^3$ to be the initial set of states, and $\mathcal{X}_{VF} = [0, 0.7]^2 \times [0, 2]$ to be the set of states that must be visited only finitely often. We then search for a CBBC as in Definition 4.1 following Algorithm 1 by using the TSSOS [32] package by setting $k_{\max} = 8$, and incrementing the value of d from $d = 1$ to $d = 8$. We find a CBBC as in Definition 3.1, of degree $d = 4$ with $k = 2$. The time taken for this is around 20 minutes and the certificates are given in Appendix C.

8. CONCLUSION

This paper advocated the notion of co-Büchi barrier certificates that act as a proof that a region is visited only finitely often. These CBBCs have been demonstrated to be useful in the automata-theoretic verification of discrete-time dynamical systems. This paper shows that techniques to compute standard barrier certificates can be extended to search for CBBCs. Some promising next steps are to understand the applicability of CBBCs in verifying quantitative and time-critical properties of dynamical systems via weighted, stochastic, and timed automata based specifications.

REFERENCES

- [1] S. Prajna, A. Papachristodoulou, P. A. Parrilo, Introducing sostools: A general purpose sum of squares programming solver, in: Proceedings of the 41st IEEE Conference on Decision and Control, 2002, pp. 741–746.
- [2] S. Schewe, B. Finkbeiner, Bounded synthesis, in: International symposium on automated technology for verification and analysis, Springer, 2007, pp. 474–488.
- [3] E. Filiot, N. Jin, J.-F. Raskin, An antichain algorithm for ltl realizability, in: International Conference on Computer Aided Verification, 2009, pp. 263–277.
- [4] E. Filiot, N. Jin, J.-F. Raskin, Antichains and compositional algorithms for ltl synthesis, Formal Methods in System Design 39 (3) (2011) 261–296.
- [5] A. Bohy, V. Bruyère, E. Filiot, N. Jin, J.-F. Raskin, Acacia+, a tool for ltl synthesis, in: International Conference on Computer Aided Verification, Springer, 2012, pp. 652–657.
- [6] T. A. Henzinger, P.-H. Ho, H. Wong-Toi, Hytech: A model checker for hybrid systems, International Journal on STTT (1997).
- [7] P. Tabuada, Verification and Control of Hybrid Systems: A Symbolic Approach, Springer Science & Business Media, 2009.
- [8] C. Baier, J.-P. Katoen, Principles of model checking, MIT press, 2008.
- [9] A. Pnueli, The temporal logic of programs, in: 18th Annual Symposium on Foundations of Computer Science, IEEE, 1977, pp. 46–57.
- [10] S. Prajna, A. Jadbabaie, Safety verification of hybrid systems using barrier certificates, in: International Workshop on Hybrid Systems: Computation and Control, 2004, pp. 477–492.
- [11] S. Prajna, A. Rantzer, Convex programs for temporal verification of nonlinear dynamical systems, SIAM Journal on Control and Optimization (2007) 999–1021.
- [12] A. D. Ames, X. Xu, J. W. Grizzle, P. Tabuada, Control barrier function based quadratic programs for safety critical systems, IEEE Transactions on Automatic Control (2016) 3861–3876.
- [13] A. Clark, Control barrier functions for stochastic systems, Automatica (2021) 109688.
- [14] S. Boyd, L. Vandenberghe, Convex optimization, Cambridge university press, 2004.
- [15] P. A. Parrilo, Semidefinite programming relaxations for semialgebraic problems, Mathematical Programming 96 (2003) 293–320.
- [16] T. Wongpiromsarn, U. Topcu, A. Lamperski, Automata theory meets barrier certificates: Temporal logic verification of nonlinear systems, IEEE Transactions on Automatic Control (2015).
- [17] R. Dimitrova, R. Majumdar, Deductive control synthesis for alternating-time logics, in: 2014 International Conference on Embedded Software (EMSOFT), IEEE, 2014, pp. 1–10.
- [18] I. Papusha, J. Fu, U. Topcu, R. M. Murray, Automata theory meets approximate dynamic programming: Optimal control with temporal logic constraints, in: 2016 IEEE 55th Conference on Decision and Control, IEEE, 2016, pp. 434–440.
- [19] P. Jagtap, S. Soudjani, M. Zamani, Temporal logic verification of stochastic systems using barrier certificates, in: ATVA, 2018, pp. 177–193.
- [20] P. Jagtap, S. Soudjani, M. Zamani, Formal synthesis of stochastic systems via control barrier certificates, IEEE Transactions on Automatic Control (2020).

- [21] M. Anand, A. Lavaei, M. Zamani, From small-gain theory to compositional construction of barrier certificates for large-scale stochastic systems, *IEEE Transactions on Automatic Control* 67 (10) (2022).
- [22] O. Kupferman, M. Y. Vardi, Model checking of safety properties, *Formal methods in system design* 19 (3) (2001) 291–314.
- [23] V. Murali, A. Trivedi, M. Zamani, Closure certificates, in: 27th ACM International Conference on Hybrid Systems: Computation and Control (to appear), 2024.
- [24] F. Galarza-Jimenez, V. Murali, M. Zamani, Compositional synthesis of controllers via co-büchi barrier certificates, in: The 8th IFAC Conference on Analysis and Design of Hybrid Systems (to appear), 2024.
- [25] D. Ajeleye, M. Zamani, Data-driven controller synthesis via co-büchi barrier certificates with formal guarantees, *IEEE Control Systems Letters* 8 (2024) 958–963.
- [26] D. Ajeleye, M. Zamani, Co-büchi control barrier certificates for stochastic control systems, *IEEE Control Systems Letters* (2024).
- [27] J. Esparza, J. Křetínský, From ltl to deterministic automata: A safrless compositional approach, in: *International Conference on Computer Aided Verification*, Springer, 2014, pp. 192–208.
- [28] S. Safra, On the complexity of ω -automata, in: *Proc. 29th IEEE Symp. Found. of Comp. Sci.*, 1988, pp. 319–327.
- [29] A. Pnueli, R. Rosner, On the synthesis of a reactive module, in: *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 1989, pp. 179–190.
- [30] K. Chatterjee, A. Gaiser, J. Křetínský, Automata with generalized rabin pairs for probabilistic model checking and ltl synthesis, in: *International Conference on Computer Aided Verification*, Springer, 2013, pp. 559–575.
- [31] T. A. Henzinger, N. Piterman, Solving games without determinization, in: *International Workshop on Computer Science Logic*, Springer, 2006, pp. 395–410.
- [32] J. Wang, V. Magron, J.-B. Lasserre, Tssos: A moment-sos hierarchy that exploits term sparsity, *SIAM Journal on optimization* 31 (1) (2021) 30–58.
- [33] A. Solar-Lezama, Program Synthesis by Sketching, PhD thesis, University of California, Berkeley, 2008.
- [34] B. Dutertre, L. d. Moura, A fast linear-arithmetic solver for dpll (t), in: *International Conference on Computer Aided Verification*, 2006, pp. 81–94.
- [35] L. d. Moura, N. Bjørner, Z3: An efficient smt solver, in: *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, 2008, pp. 337–340.
- [36] S. Gao, S. Kong, E. M. Clarke, dReal: An SMT solver for nonlinear theories over the reals, in: *Automated Deduction – CADE-24, Lecture Notes in Computer Science*, 2013, pp. 208–214.
- [37] D. C. Kozen, *Theory of computation*, Vol. 121, Springer, 2006.
- [38] M. Anand, V. Murali, A. Trivedi, M. Zamani, k-inductive barrier certificates for stochastic systems, in: 25th ACM International Conference on Hybrid Systems: Computation and Control, 2022, pp. 1–11.

APPENDIX A. A CONSTRUCTIVE PROOF FOR SUBSUMPTION OF THE STATE-TRIPLET APPROACH

A.1. Our Construction. We now show that our approach generalizes the earlier state triplet approach. Consider the NBA $\mathcal{A}'_b = (\Sigma, Q', Q_0, \delta', Q_{Acc})$, and let us assume that there exist m barrier certificates $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_m$ associated with state triplets (q_i, q'_i, q''_i) , (or edge pairs (e_i, e'_i)) for each $1 \leq i \leq m$, such that $q'_i \in \delta'(q_i, a_i)$ and $q''_i \in \delta'(q'_i, b_i)$ that act as a proof certificate that no trace of the system is in $L(\mathcal{A}'_b)$.

First, for convenience we assume that there are no edge pairs (e_i, e'_i) and (e_j, e'_j) for all $1 \leq i, j \leq m$, $i \neq j$, such that the edges e_i and e_j are incoming edges to the same state and the edges e'_i and e'_j are outgoing edges from the same state, *i.e.*, there are no two distinct state triplets (q_i, q'_i, q''_i) and (q_j, q'_j, q''_j) which share a common middle element ($q'_i \neq q'_j$ for all $0 \leq i, j \leq m$, $i \neq j$). Situations which may arise are based on the example shown in Figure 5. To consider a single triplet, we replace their barrier certificates, with a new one by taking either the minimum or maximum of the barrier certificates $\mathcal{B}_i$ and $\mathcal{B}_j$ or neither. We list three different cases as follows:

- Suppose we have the edges e_i and e_j to be the same edge and the edges e'_i and e'_j to be different. In this case we have $a_i = a_j$. We know that for any $x \in \mathcal{X}$ with $\mathcal{L}(x) = a_i$, we have both $\mathcal{B}_i(x) \leq 0$ and $\mathcal{B}_j(x) \leq 0$, and at least one of $\mathcal{B}_i(x) > 0$ or $\mathcal{B}_j(x) > 0$ for all states $x \in \mathcal{X}$ such that either $\mathcal{L}(x) = b_i$ or $\mathcal{L}(x) = b_j$. We therefore consider $\mathcal{B}'(x) = \max\{\mathcal{B}_i(x), \mathcal{B}_j(x)\}$ as the new barrier certificate for both the edge pairs.
- Suppose we have the edges e'_i and e'_j to be the same edge and the edges e_i and e_j to be different. In this case we have $b_i = b_j$. We know that for any $x \in \mathcal{X}$ with $\mathcal{L}(x) = a_i$, we have at least one of $\mathcal{B}_i(x) \leq 0$ or $\mathcal{B}_j(x) \leq 0$, and both $\mathcal{B}_i(x) > 0$ and $\mathcal{B}_j(x) > 0$ for all states $x \in \mathcal{X}$ with $\mathcal{L}(x) = b_i$. We therefore consider $\mathcal{B}'(x) = \min\{\mathcal{B}_i(x), \mathcal{B}_j(x)\}$ as the new barrier certificate for both the edge pairs.

- Neither edges are the same but they are incident on the same vertex, *i.e.*, the states q'_i and q'_j are the same for both of the triplets. In this case we do not consider the barrier certificate as both paths continue on and must be cut at some future point, namely there is no barrier certificate between e_i and e'_j nor is there one between e_j and e'_i and so there must be some future state triplet or edge pairs which disallow paths starting from e'_i and e'_j .

Now, we divide the states of NBA $\mathcal{A}'_b$ into two sets Q'_l or Q'_r based on whether the state is to the left or the right of *some* triplet. If a state $q \in Q'$ is to the left of, or is, the middle element of some state triplet along a path from an initial state to an accepting state, then it is in Q'_l , else it is in Q'_r . Observe that all accepting states must be in Q'_r as there is some triplet for every path by our assumption.

We now define a CBBC $\mathcal{B}$ as in Definition 4.1, with the counter value $k = 0$, over the synchronized product as follows. First, for all states $x \in \mathcal{X}$ and every state $q_l \in Q'_l$, we define $\mathcal{B}(x, l, 0)$ as the maximum value of the barriers $\mathcal{B}_1, \dots, \mathcal{B}_m$ over the regions. For all states $x \in \mathcal{X}$ and $q_r \in Q'_r$, we define $\mathcal{B}(x, r, 0)$ and $\mathcal{B}(x, r, 1)$ as the minimum value of the barrier certificates $\mathcal{B}_1, \dots, \mathcal{B}_m$ over the regions $b_1, \dots, b_m$. In summary, we get:

$$\mathcal{B}(x, \xi, \ell) = \begin{cases} \max_{1 \leq i \leq m} \left\{ \max_{\substack{x \in \mathcal{X} \\ \mathcal{L}(x) = a_i}} \{\mathcal{B}_i(x)\} \right\} & \text{for all } q_\xi \in Q'_l, \ell = 0 \\ \min_{1 \leq i \leq m} \left\{ \min_{\substack{x \in \mathcal{X} \\ \mathcal{L}(x) = b_i}} \{\mathcal{B}_i(x)\} \right\}, & \text{for all } q_\xi \in Q'_r, \ell \in \{0, 1\}. \end{cases}$$

Observe that the largest value this achieves for all states $q'_l \in Q'_l$ is still less than 0 and so we satisfy condition (9). Next, the value of the CBBC for all states $q'_r \in Q'_r$ is greater than 0 and so we satisfy condition (3) for all states $q' \in Q_{Acc}$.

Conditions (11) and (12) hold within the sets Q'_l and Q'_r as they are constants in the two partitions. Secondly, we observe that no state in Q'_r can be reached by any trace of the system as all the state triplets have been cut, and so there exists no state $x \in \mathcal{X}$ and $q_i \in Q'_l$ such that there exists a state $q'_j \in Q'_r$ with $q'_j \in \delta(q_i, \mathcal{L}(x))$. This guarantees conditions (11) and (12) hold for every state $x \in \mathcal{X}$ and $q \in Q$. We illustrate this partition of the states of the NBA $\mathcal{A}'_b$ in Figure 6, where the states q'_i denote the middle state of the triplets.

Observe that the CBBC can be defined as a piecewise function which is fixed to a constant value less than 0 for all states $q_l \in Q'_l$ and $x \in \mathcal{X}$, and to a constant value greater than 0 for all states $q_r \in Q'_r$ and states $x \in \mathcal{X}$.

APPENDIX B. CBBC FOR SECTION 7.2

The CBBC $\mathcal{B}_{\ell,i}(x)$ are defined picewise over $\ell \in Q$ ($\ell \in [0..2]$), and $i \in [0..3]$ as:

$$\begin{aligned} : \mathcal{B}_{0,0}(x, y) = & -0.001479 \cdot x^6 - 0.012757 \cdot x^5 \cdot y + 0.05985 \cdot x^4 \cdot y^2 - 0.075517 \cdot x^3 \cdot y^3 + 0.071967 \cdot x^2 \cdot y^4 - \\ & 0.042324 \cdot x \cdot y^5 + 0.010623 \cdot y^6 - 0.003189 \cdot x^5 - 0.011026 \cdot x^4 \cdot y - 0.004925 \cdot x^3 \cdot y^2 - 0.015355 \cdot x^2 \cdot \end{aligned}$$

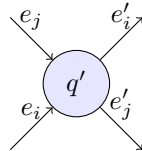


FIGURE 5. Corner cases where multiple barrier certificates are present for the same intermediate state. In all cases we can reduce this to a single barrier certificate over the state q' . Here we can either replace the barrier certificates with the maximum or minimum or ignore these.

$$y^3 + 0.00277 \cdot x \cdot y^4 - 0.005104 \cdot y^5 + 0.117479 \cdot x^4 - 0.022629 \cdot x^3 \cdot y - 0.151256 \cdot x^2 \cdot y^2 + 0.299684 \cdot x \cdot y^3 - 0.037504 \cdot y^4 + 0.017933 \cdot x^3 + 0.061936 \cdot x^2 \cdot y + 0.005285 \cdot x \cdot y^2 + 0.030286 \cdot y^3 - 1.727458 \cdot x^2 + 0.393963 \cdot x \cdot y - 1.348065 \cdot y^2 + 1.0e - 6 \cdot x - 1.0e - 6 \cdot y + 6.101348$$

$$: \mathcal{B}_{0,1}(x, y) := 0.014904 \cdot x^6 - 0.001828 \cdot x^5 \cdot y + 0.103405 \cdot x^4 \cdot y^2 - 0.079955 \cdot x^3 \cdot y^3 + 0.109044 \cdot x^2 \cdot y^4 - 0.047584 \cdot x \cdot y^5 + 0.017908 \cdot y^6 + 0.001017 \cdot x^5 - 0.00713 \cdot x^4 \cdot y + 0.005648 \cdot x^3 \cdot y^2 - 0.018626 \cdot x^2 \cdot y^3 + 0.010244 \cdot x \cdot y^4 - 0.006376 \cdot y^5 - 0.014842 \cdot x^4 - 0.130578 \cdot x^3 \cdot y - 0.369789 \cdot x^2 \cdot y^2 + 0.293624 \cdot x \cdot y^3 - 0.110469 \cdot y^4 - 0.004996 \cdot x^3 + 0.044031 \cdot x^2 \cdot y - 0.025907 \cdot x \cdot y^2 + 0.035927 \cdot y^3 - 1.60888 \cdot x^2 + 0.682265 \cdot x \cdot y - 1.300488 \cdot y^2 + 1.0e - 6 \cdot x + 1.926749$$

$$: \mathcal{B}_{0,2}(x, y) := 0.020866 \cdot x^6 + 0.0065 \cdot x^5 \cdot y + 0.114056 \cdot x^4 \cdot y^2 - 0.072159 \cdot x^3 \cdot y^3 + 0.114243 \cdot x^2 \cdot y^4 - 0.044565 \cdot x \cdot y^5 + 0.018515 \cdot y^6 + 0.004629 \cdot x^5 - 0.004803 \cdot x^4 \cdot y + 0.013346 \cdot x^3 \cdot y^2 - 0.01922 \cdot x^2 \cdot y^3 + 0.013768 \cdot x \cdot y^4 - 0.006355 \cdot y^5 - 0.045795 \cdot x^4 - 0.208793 \cdot x^3 \cdot y - 0.383569 \cdot x^2 \cdot y^2 + 0.245491 \cdot x \cdot y^3 - 0.106145 \cdot y^4 - 0.024058 \cdot x^3 + 0.031517 \cdot x^2 \cdot y - 0.045583 \cdot x \cdot y^2 + 0.036307 \cdot y^3 - 1.673498 \cdot x^2 + 0.866638 \cdot x \cdot y - 1.407303 \cdot y^2 + 1.0e - 6 \cdot x + 1.364599$$

$$: \mathcal{B}_{0,3}(x) := 0.014596 \cdot x^6 + 0.014443 \cdot x^5 \cdot y + 0.091242 \cdot x^4 \cdot y^2 - 0.05221 \cdot x^3 \cdot y^3 + 0.092222 \cdot x^2 \cdot y^4 - 0.034871 \cdot x \cdot y^5 + 0.0139 \cdot y^6 + 0.003293 \cdot x^5 - 0.004216 \cdot x^4 \cdot y + 0.009595 \cdot x^3 \cdot y^2 - 0.015104 \cdot x^2 \cdot y^3 + 0.010328 \cdot x \cdot y^4 - 0.004994 \cdot y^5 + 0.030812 \cdot x^4 - 0.280669 \cdot x^3 \cdot y - 0.216901 \cdot x^2 \cdot y^2 + 0.16154 \cdot x \cdot y^3 - 0.044066 \cdot y^4 - 0.017154 \cdot x^3 + 0.026044 \cdot x^2 \cdot y - 0.03371 \cdot x \cdot y^2 + 0.029005 \cdot y^3 - 1.956452 \cdot x^2 + 1.019239 \cdot x \cdot y - 1.657867 \cdot y^2 + 2.587095$$

$$: \mathcal{B}_{1,0}(x, y) := 1.169368 \cdot x^6 - 0.802076 \cdot x^5 \cdot y + 1.61694 \cdot x^4 \cdot y^2 - 0.806867 \cdot x^3 \cdot y^3 + 3.620299 \cdot x^2 \cdot y^4 - 1.427064 \cdot x \cdot y^5 + 1.50265 \cdot y^6 - 7.151424 \cdot x^5 + 3.095272 \cdot x^4 \cdot y - 8.119412 \cdot x^3 \cdot y^2 + 1.904013 \cdot x^2 \cdot y^3 - 15.292198 \cdot x \cdot y^4 + 4.045883 \cdot y^5 + 9.863108 \cdot x^4 - 0.909527 \cdot x^3 \cdot y + 9.274002 \cdot x^2 \cdot y^2 - 0.648806 \cdot x \cdot y^3 + 8.101657 \cdot y^4 + 3.193949 \cdot x^3 - 2.674896 \cdot x^2 \cdot y + 5.269647 \cdot x \cdot y^2 - 2.075598 \cdot y^3 - 0.355375 \cdot x^2 + 0.248207 \cdot x \cdot y + 0.604798 \cdot y^2 - 0.109638 \cdot x - 0.235628 \cdot y + 4.284889$$

$$: \mathcal{B}_{1,1}(x, y) := 1.079006 \cdot x^6 - 0.514729 \cdot x^5 \cdot y + 3.468024 \cdot x^4 \cdot y^2 + 3.107501 \cdot x^3 \cdot y^3 + 11.825979 \cdot x^2 \cdot y^4 + 9.156119 \cdot x \cdot y^5 + 4.958857 \cdot y^6 - 6.65663 \cdot x^5 + 2.319631 \cdot x^4 \cdot y - 11.363004 \cdot x^3 \cdot y^2 + 2.422806 \cdot x^2 \cdot y^3$$

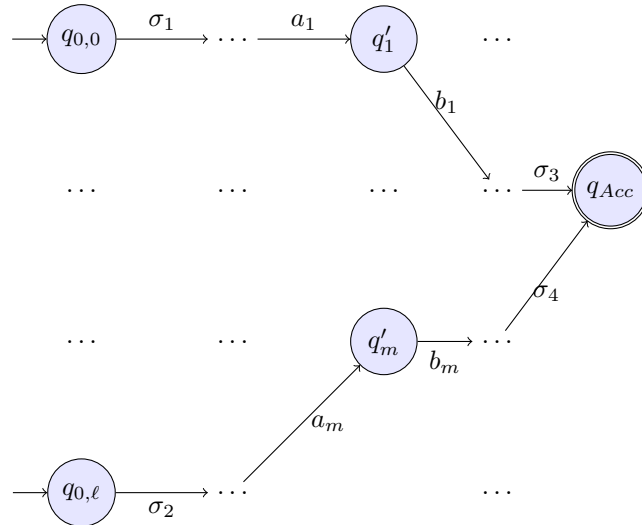


FIGURE 6. The NBA $\mathcal{A}'_b$ is divided along the middle elements of the state triplets. The states to the left of (and including) the states q'_i , for all $1 \leq i \leq m$, are in Q'_l and the right are in Q'_r .

$$\begin{aligned}
& y^3 - 5.018148 \cdot x \cdot y^4 + 6.555657 \cdot y^5 + 9.146765 \cdot x^4 - 2.107326 \cdot x^3 \cdot y + 4.292546 \cdot x^2 \cdot y^2 - 5.384374 \cdot x \cdot y^3 + 4.530556 \cdot y^4 + 3.30525 \cdot x^3 - 1.60986 \cdot x^2 \cdot y + 2.758191 \cdot x \cdot y^2 - 2.695313 \cdot y^3 - 0.477553 \cdot x^2 + 0.939179 \cdot x \cdot y + 0.13928 \cdot y^2 + 0.069863 \cdot x + 0.026055 \cdot y + 3.61205 \\
& : \mathcal{B}_{1,2}(x, y) := 0.8925 \cdot x^6 - 0.742628 \cdot x^5 \cdot y + 3.15367 \cdot x^4 \cdot y^2 + 3.382412 \cdot x^3 \cdot y^3 + 12.385971 \cdot x^2 \cdot y^4 + 9.857827 \cdot x \cdot y^5 + 5.144468 \cdot y^6 - 5.872276 \cdot x^5 + 2.954834 \cdot x^4 \cdot y - 10.317188 \cdot x^3 \cdot y^2 + 2.308949 \cdot x^2 \cdot y^3 - 4.777581 \cdot x \cdot y^4 + 6.499851 \cdot y^5 + 8.932045 \cdot x^4 - 1.896671 \cdot x^3 \cdot y + 4.762227 \cdot x^2 \cdot y^2 - 5.233601 \cdot x \cdot y^3 + 4.614206 \cdot y^4 + 2.101369 \cdot x^3 - 1.36296 \cdot x^2 \cdot y + 2.356684 \cdot x \cdot y^2 - 2.4981 \cdot y^3 - 0.911951 \cdot x^2 + 1.140637 \cdot x \cdot y - 0.025658 \cdot y^2 + 0.023114 \cdot x + 0.047989 \cdot y + 4.729123 \\
& : \mathcal{B}_{1,3}(x, y) := 0.657481 \cdot x^6 + 0.481838 \cdot x^5 \cdot y + 2.914344 \cdot x^4 \cdot y^2 + 2.589529 \cdot x^3 \cdot y^3 + 5.201449 \cdot x^2 \cdot y^4 + 2.90154 \cdot x \cdot y^5 + 2.630666 \cdot y^6 - 4.665976 \cdot x^5 - 1.729505 \cdot x^4 \cdot y - 12.526088 \cdot x^3 \cdot y^2 - 5.730965 \cdot x^2 \cdot y^3 - 7.31455 \cdot x \cdot y^4 + 7.158644 \cdot y^5 + 10.201324 \cdot x^4 - 1.176185 \cdot x^3 \cdot y + 12.251171 \cdot x^2 \cdot y^2 + 3.196583 \cdot x \cdot y^3 + 13.482927 \cdot y^4 - 7.195209 \cdot x^3 + 4.947589 \cdot x^2 \cdot y - 2.009166 \cdot x \cdot y^2 + 1.422074 \cdot y^3 + 2.104938 \cdot x^2 + 2.740789 \cdot x \cdot y + 2.251575 \cdot y^2 - 0.352426 \cdot x + 0.93496 \cdot y + 6.437098 \\
& : \mathcal{B}_{2,0}(x, y) := 6.647834 \cdot x^6 - 23.026921 \cdot x^5 \cdot y + 63.958752 \cdot x^4 \cdot y^2 - 47.22306 \cdot x^3 \cdot y^3 + 21.22077 \cdot x^2 \cdot y^4 - 16.818629 \cdot x \cdot y^5 + 20.858006 \cdot y^6 - 17.291677 \cdot x^5 - 21.229591 \cdot x^4 \cdot y - 31.082763 \cdot x^3 \cdot y^2 - 18.195494 \cdot x^2 \cdot y^3 - 25.085691 \cdot x \cdot y^4 - 5.694037 \cdot y^5 + 29.391701 \cdot x^4 + 28.493682 \cdot x^3 \cdot y + 35.15935 \cdot x^2 \cdot y^2 - 2.539602 \cdot x \cdot y^3 + 64.361465 \cdot y^4 + 5.216234 \cdot x^3 + 6.999094 \cdot x^2 \cdot y + 3.81613 \cdot x \cdot y^2 + 6.50213 \cdot y^3 + 13.044233 \cdot x^2 + 2.41708 \cdot x \cdot y + 16.618755 \cdot y^2 + 0.582513 \cdot x + 2.187375 \cdot y + 15.750095 \\
& : \mathcal{B}_{2,1}(x, y) := 2.105094 \cdot x^6 - 1.453849 \cdot x^5 \cdot y + 35.178515 \cdot x^4 \cdot y^2 - 49.88313 \cdot x^3 \cdot y^3 + 45.685832 \cdot x^2 \cdot y^4 - 4.629385 \cdot x \cdot y^5 + 9.011891 \cdot y^6 - 13.536015 \cdot x^5 - 24.339273 \cdot x^4 \cdot y - 31.687374 \cdot x^3 \cdot y^2 - 11.876397 \cdot x^2 \cdot y^3 - 16.015567 \cdot x \cdot y^4 - 2.549349 \cdot y^5 + 25.993643 \cdot x^4 + 18.066015 \cdot x^3 \cdot y + 24.268005 \cdot x^2 \cdot y^2 + 11.100776 \cdot x \cdot y^3 + 53.216611 \cdot y^4 + 3.59729 \cdot x^3 + 5.936538 \cdot x^2 \cdot y + 3.004211 \cdot x \cdot y^2 + 5.123685 \cdot y^3 + 11.34886 \cdot x^2 + 2.645745 \cdot x \cdot y + 13.059074 \cdot y^2 + 0.797674 \cdot x + 1.733336 \cdot y + 13.661955 \\
& : \mathcal{B}_{2,2}(x, y) := 1.722178 \cdot x^6 + 5.310932 \cdot x^5 \cdot y + 18.349052 \cdot x^4 \cdot y^2 - 31.02038 \cdot x^3 \cdot y^3 + 53.695332 \cdot x^2 \cdot y^4 - 8.244383 \cdot x \cdot y^5 + 2.053936 \cdot y^6 - 12.174742 \cdot x^5 - 25.506336 \cdot x^4 \cdot y - 24.443983 \cdot x^3 \cdot y^2 - 6.502687 \cdot x^2 \cdot y^3 - 11.222995 \cdot x \cdot y^4 - 0.641161 \cdot y^5 + 19.955712 \cdot x^4 + 10.419893 \cdot x^3 \cdot y + 20.713391 \cdot x^2 \cdot y^2 + 21.301025 \cdot x \cdot y^3 + 42.607373 \cdot y^4 + 2.265752 \cdot x^3 + 5.151999 \cdot x^2 \cdot y + 2.673623 \cdot x \cdot y^2 + 4.233808 \cdot y^3 + 9.995201 \cdot x^2 + 2.756518 \cdot x \cdot y + 9.961636 \cdot y^2 + 1.05112 \cdot x + 1.362802 \cdot y + 11.696231 \\
& : \mathcal{B}_{2,3}(x, y) := 1.830446 \cdot x^6 + 7.207784 \cdot x^5 \cdot y + 8.603092 \cdot x^4 \cdot y^2 - 10.95115 \cdot x^3 \cdot y^3 + 44.535124 \cdot x^2 \cdot y^4 - 15.92115 \cdot x \cdot y^5 - 3.079411 \cdot y^6 - 10.209742 \cdot x^5 - 23.847993 \cdot x^4 \cdot y - 14.326739 \cdot x^3 \cdot y^2 - 3.364347 \cdot x^2 \cdot y^3 - 7.604397 \cdot x \cdot y^4 + 0.614566 \cdot y^5 + 12.955935 \cdot x^4 + 8.069967 \cdot x^3 \cdot y + 22.039468 \cdot x^2 \cdot y^2 + 27.849717 \cdot x \cdot y^3 + 33.681351 \cdot y^4 + 0.812285 \cdot x^3 + 5.228008 \cdot x^2 \cdot y + 2.414385 \cdot x \cdot y^2 + 3.676612 \cdot y^3 + 8.583767 \cdot x^2 + 2.504491 \cdot x \cdot y + 7.086297 \cdot y^2 + 1.186848 \cdot x + 0.995341 \cdot y + 9.732926
\end{aligned}$$

APPENDIX C. CBBC FOR SECTION 7.3

The CBBC $\mathcal{B}_{\ell,i}(x)$ are defined piecewise over and $i \in [0..2]$ as:

$$\begin{aligned}
& : \mathcal{B}_0(x, y, z) = -0.110879 \cdot x^4 - 0.092034 \cdot x^3 \cdot y - 0.082008 \cdot x^3 \cdot z + 0.109202 \cdot x^2 \cdot y^2 - 1.7e - 5 \cdot x^2 \cdot y \cdot z + 0.061129 \cdot x^2 \cdot z^2 - 0.091925 \cdot x \cdot y^3 - 4.0e - 6 \cdot x \cdot y^2 \cdot z + 3.4e - 5 \cdot x \cdot y \cdot z^2 - 0.082014 \cdot x \cdot z^3 - 0.127569 \cdot y^4 - 0.025422 \cdot y^3 \cdot z - 0.022908 \cdot y^2 \cdot z^2 - 0.025878 \cdot y \cdot z^3 - 0.097348 \cdot z^4 + 1.774397 \cdot x^3 - 0.000857 \cdot x^2 \cdot y + 0.000538 \cdot x^2 \cdot z - 0.000318 \cdot x \cdot y^2 - 0.001115 \cdot x \cdot y \cdot z + 0.000179 \cdot x \cdot z^2 + 1.827821 \cdot y^3 - 0.000432 \cdot y^2 \cdot z - 0.000194 \cdot y \cdot z^2 + 1.632205 \cdot z^3 + 0.089029 \cdot x^2 + 0.706954 \cdot x \cdot y + 0.616631 \cdot x \cdot z + 0.301684 \cdot y^2 + 0.207754 \cdot y \cdot z + 0.287124 \cdot z^2 - 10.00198 \cdot x - 10.300025 \cdot y - 9.201995 \cdot z - 0.322743
\end{aligned}$$

$$\begin{aligned}
&: \mathcal{B}_1(x, y, z) = -0.286681 \cdot x^4 - 0.208624 \cdot x^3 \cdot y - 0.205306 \cdot x^3 \cdot z + 0.282097 \cdot x^2 \cdot y^2 - 0.000111 \cdot x^2 \cdot y \cdot z \\
&\quad + 0.174779 \cdot x^2 \cdot z^2 - 0.208234 \cdot x \cdot y^3 + 5.8e - 5 \cdot x \cdot y^2 \cdot z + 3.2e - 5 \cdot x \cdot y \cdot z^2 - 0.205275 \cdot x \cdot z^3 - \\
&\quad 0.343453 \cdot y^4 - 0.02198 \cdot y^3 \cdot z - 0.107036 \cdot y^2 \cdot z^2 - 0.025665 \cdot y \cdot z^3 - 0.193375 \cdot z^4 + 4.496906 \cdot x^3 - \\
&\quad 0.002283 \cdot x^2 \cdot y + 0.001331 \cdot x^2 \cdot z - 0.001123 \cdot x \cdot y^2 - 0.002579 \cdot x \cdot y \cdot z + 0.00018 \cdot x \cdot z^2 + 4.692951 \cdot y^3 \\
&\quad - 0.001385 \cdot y^2 \cdot z + 2.6e - 5 \cdot y \cdot z^2 + 3.083638 \cdot z^3 + 0.199697 \cdot x^2 + 1.606444 \cdot x \cdot y + 1.543309 \cdot x \cdot z \\
&\quad + 0.921929 \cdot y^2 + 0.212692 \cdot y \cdot z + 0.586415 \cdot z^2 - 25.347415 \cdot x - 26.445779 \cdot y - 17.384738 \cdot z + 36.288487 \\
\\
&: \mathcal{B}_2(x, y, z) = -0.310531 \cdot x^4 - 0.192396 \cdot x^3 \cdot y - 0.192537 \cdot x^3 \cdot z + 0.253625 \cdot x^2 \cdot y^2 - 0.000124 \cdot x^2 \cdot y \cdot z \\
&\quad + 0.140886 \cdot x^2 \cdot z^2 - 0.191943 \cdot x \cdot y^3 + 8.2e - 5 \cdot x \cdot y^2 \cdot z + 2.5e - 5 \cdot x \cdot y \cdot z^2 - 0.192503 \cdot x \cdot z^3 - \\
&\quad 0.371679 \cdot y^4 + 0.001279 \cdot y^3 \cdot z - 0.158635 \cdot y^2 \cdot z^2 - 0.002735 \cdot y \cdot z^3 - 0.20944 \cdot z^4 + 4.788563 \cdot x^3 - \\
&\quad 0.00241 \cdot x^2 \cdot y + 0.001301 \cdot x^2 \cdot z - 0.001302 \cdot x \cdot y^2 - 0.002351 \cdot x \cdot y \cdot z + 8.0e - 5 \cdot x \cdot z^2 + 5.016419 \cdot y^3 \\
&\quad - 0.001557 \cdot y^2 \cdot z + 6.3e - 5 \cdot y \cdot z^2 + 3.264085 \cdot z^3 + 0.405335 \cdot x^2 + 1.486829 \cdot x \cdot y + 1.447237 \cdot x \cdot z + \\
&\quad 1.175779 \cdot y^2 + 0.04123 \cdot y \cdot z + 0.806732 \cdot z^2 - 26.991353 \cdot x - 28.26892 \cdot y - 18.40208 \cdot z + 51.574377
\end{aligned}$$

¹UNIVERSITY OF COLORADO BOULDER

Email address: {vishnu.murali,ashutosh.trivedi,majid.zamani}@colorado.edu